KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad – 500029

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAB MANUAL

SKILL DEVELOPMENT COURSE 3

B.Tech. IV YEAR I SEM (KR23)
ACADEMIC YEAR 2026-27

KESHAV MEMORIAL INSTITUTE OF TECHNOLO

(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad

Certificate

This is to certify that following is a Bonafide Record of the workbook task done by

_____bearing Roll No_____of

_____Branch of_____year B.Tech Course in the

_____Subject during the Academic year_____&_____ under our supervision.

Number of experiments completed:_____

Signature of Internal Examiner

Signature of Head of the Dept.

INDEX

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTE)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad – 500029

Department of Computer Science & Engineering

Vision of the Institution:

To be the fountain head of latest technologies, producing highly skilled, globally competent engineers.

Mission of the Institution:

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi modal platforms and prepare students to become successful professionals.
- To establish Industry Institute Interaction to make students ready for the industry.
- To provide exposure to students on latest hardware and software tools.
- To promote research based projects/activities in the emerging areas of technology convergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTE)

Accredited by NBA & NAAC, Approved by
AICTE, Affiliated to JNTUH, Narayanguda,
Hyderabad – 500029

Department of Computer Science & Engineering

Vision of the Department:

To be among the region's premier teaching and research Computer Science and Engineering departments producing globally competent and socially responsible graduates in the most conducive academic environment.

Mission of the Department:

- To provide faculty with state-of-the-art facilities for continuous professional development and research, both in foundational aspects and of relevance to emerging computing trends.
- To impart skills that transform students to develop technical solutions for societal needs and inculcate entrepreneurial talents.
- To inculcate an ability in students to pursue the advancement of knowledge in various specializations of Computer Science and Engineering and make them industry-ready.
- To engage in collaborative research with academia and industry and generate adequate resources for research activities for seamless transfer of knowledge resulting in sponsored projects and consultancy.
- To cultivate responsibility through sharing of knowledge and innovative computing solutions that benefits the society-at-large.
- To collaborate with academia, industry and community to set high standards in academic excellence and in fulfilling societal responsibilities.

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTE)
Accredited by NBA & NAAC, Approved by
AICTE, Affiliated to JNTUH, Narayanguda,
Hyderabad – 500029

Department of Computer Science & Engineering

PROGRAM OUTCOMES (POs)

PO1: Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6: The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTE)
Accredited by NBA & NAAC, Approved by
AICTE, Affiliated to JNTUH, Narayanguda,
Hyderabad – 500029

Department of Computer Science & Engineering

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: An ability to analyze the common business functions to design and develop appropriate Computer Science solutions for social upliftment.

PSO2: Shall have expertise on the evolving technologies like Python, Machine Learning, Deep Learning, Internet of Things (IOT), Data Science, Full stack development, Social Networks, Cyber Security, Big Data, Mobile Apps, CRvM, ERP etc.

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTE)
Accredited by NBA & NAAC, Approved by
AICTE, Affiliated to JNTUH, Narayanguda,
Hyderabad – 500029

Department of Computer Science & Engineering

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Graduates will have successful careers in computer related engineering fields or will be able to successfully pursue advanced higher education degrees.

PEO2: Graduates will try and provide solutions to challenging problems in their profession by applying computer engineering principles.

PEO3: Graduates will engage in life-long learning and professional development by rapidly adapting changing work environment.

PEO4: Graduates will communicate effectively, work collaboratively and exhibit high levels of professionalism and ethical responsibility.

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTE)

Accredited by NBA & NAAC, Approved by
AICTE, Affiliated to JNTUH, Narayanguda,
Hyderabad – 500029

Department of Computer Science & Engineering

B.Tech. IV Year I Semester Course Syllabus (KR23)
SKILL DEVELOPMENT COURSE 3(23CS705PC)

Prerequisite/ Corequisite:

1. 23CS503PC - Web Technologies Course

Course Objectives: The course will help to

1. Provide an understanding of the fundamentals of ReactJS for developing modern web applications.
2. Explain the concepts of JSX, components, props, and state for building reusable user interface components.
3. Enable students to use React Hooks for effective state management and handling side effects.
4. Introduce client-side routing techniques using React Router for single-page applications.
5. Develop the ability to build interactive, data-driven applications using forms, conditional rendering, and API integration.

Course Outcomes: After learning the concepts of the course, the student is able to

1. Understand the core concepts of ReactJS and develop applications using JSX and functional components.
2. Apply props and state to manage data flow and user interactions in React applications.
3. Use React Hooks such as useState and useEffect to manage component state and lifecycle behavior.
4. Implement routing and navigation in single-page applications using React Router.
5. Design and develop complete ReactJS applications that consume external APIs and handle user input effectively.

LIST OF EXERCISES

1.Build a React application to demonstrate JSX and Virtual DOM

2.Create a product listing page using multiple stateless React components. The page should include a header, product cards displaying name, price, and category, and a "Buy Now" button that responds to user interactions using props.

3.Develop an inventory management application using React Hooks such as useState, use Effect, useContext, useRef, and useMemo. The application should allow adding products, searching products, calculating total inventory value, and toggling between light and dark themes dynamically.

4.Build a simplified e-commerce application using React Router. The application should include Home, Products, Product Details, Contact, Login, Cart, and 404 pages with dynamic routing, protected routes, and navigation links.

5.Create a React form where input fields are dynamically generated from a JavaScript configuration object. The form should store user input using state and display the submitted data below the form.

6.Develop a React single-page application with routing between Home and Users pages. Fetch user data from a public API using useEffect and useState, display the data conditionally, and show a loading indicator while data is being fetched.

TEXT BOOKS:

1.Learning React, Alex Banks, Eve Porcello, O'Reilly Media.
2. React Up & Running, Stoyan Stefanov, O'Reilly Media.

REFERENCE BOOKS:

1.Full stack React, Anthony Accomazzo et al., Fullstack.io.
2.Pro React, Adam Freeman, Apress.
3.The Road to Learn React, Robin Wieruch.

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTE)
Accredited by NBA & NAAC, Approved by AICTE,
Affiliated to JNTUH, Narayanguda, Hyderabad – 500029

Department of Computer Science & Engineering

Course Outcomes and CO-PO-PSO Mapping

Course Outcomes:

After learning the contents of this course, the student is able to

CO1	Understand the core concepts of ReactJS and develop applications using JSX and functional components.
CO2	Apply props and state to manage data flow and user interactions in React applications.
CO3	Use React Hooks such as useState and useEffect to manage component state and lifecycle behavior.
CO4	Implement routing and navigation in single-page applications using React Router.
CO5	Design and develop complete ReactJS applications that consume external APIs and handle user input effectively.

CO-PO-PSO MAPPING:

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	2	2	–	3	–	–	–	–	1	–	1	2	3
CO2	3	3	3	–	3	–	–	–	1	1	–	1	2	3
CO3	3	3	3	1	3	–	–	–	1	1	–	2	2	3
CO4	3	2	3	–	3	–	–	–	1	1	–	1	2	3
CO5	3	3	3	2	3	–	–	–	2	2	1	2	3	3

Installation Steps

Step 1: Install Node.js

Download the LTS version from:

https://nodejs.org

Verify the installation.

Open Command Prompt and type:

node -v

Step 2: Install Visual Studio Code

Download from:

https://code.visualstudio.com

Step 3: Create a React Project

Open Command Prompt.

Go to the folder where you want to create the project.

[On my system:

E:\newreactjsprogram>node -v

v22.11.0

E:\newreactjsprogram>npm -v

10.9.0

E:\newreactjsprogram>]

from this folder:

npx create-react-app hello-react

This downloads React and creates the project.

after 2 or 3 mins

....

We suggest that you begin by typing:

cd hello-react

npm start

Happy hacking!

E:\newreactjsprogram>

.....

Step 4: Open the Project

Move into the project folder:

E:\newreactjsprogram>cd hello-react

E:\newreactjsprogram\hello-react>

Open it in VS Code using code .

Step 5: Project Structure

hello-react

¦

+-- node_modules

+-- public

¦ +-- index.html

¦

+-- src

¦ +-- App.js

¦ +-- App.css

¦ +-- index.js

¦

+-- package.json

+-- package-lock.json

Step 6: Replace App.js

Open

src\App.js

Delete everything and write:

```
function App() {
  return (
    <div>
      <h1>Hello World!</h1>
      <p>Welcome to ReactJS.</p>
    </div>
  );
```

}

export default App;

Save the file.

Step 7: Run the Application

Open Command Prompt inside the project folder.

E:\newreactjsprogram\hello-react

npm start

After a few seconds you will see something like:

Compiled successfully!

You can now view hello-react in the browser.

Local:

http://localhost:3000

Important Files

File	Purpose
App.jsx	Main React component
main.jsx	Entry point of the application
index.html	Main HTML page
package.json	Project configuration and dependencies

Commands Summary

```
# Create project
npm create vite@latest jsx-demo -- --template react

# Open project
cd jsx-demo

# Install dependencies
npm install

# Start application
npm run dev
```

1.Build a React application to demonstrate JSX and Virtual DOM

Installation:

npm create vite@latest jsx-demo -- --template react
cd jsx-demo
npm install
npm run dev

Aim

To build a React application that demonstrates JSX and Virtual DOM concepts.

Software Requirements

- Node.js
- React.js
- Visual Studio Code
- Web Browser

Theory

JSX (JavaScript XML) allows writing HTML-like syntax within JavaScript. React uses a Virtual DOM to efficiently update only the changed parts of the user interface, improving performance.

Algorithm

1. Create a React application.
2. Create a state variable using useState.
3. Display the state value using JSX.
4. Add a button to update the state.
5. Observe the UI update through Virtual DOM.

Program

```jsx
import React, { useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>JSX and Virtual DOM Demo</h1>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}

export default App;
```

Output

- Displays count value.
- Count updates instantly when button is clicked.

Sample Input

Click the **Increment** button.

Sample Output

JSX and Virtual DOM Demo

Count: 0

[Increment]

After Click:

Count: 1

Result

Thus, a React application demonstrating JSX and Virtual DOM was developed and executed successfully.

2. Product Listing Page Using React Components

Aim

1. To create reusable React components.
2. To display product information using props.
3. To understand component-based architecture in React.
4. To implement event handling using buttons.

Software Requirements

- Windows 10/11
- Node.js
- React.js
- Visual Studio Code
- Google Chrome

Theory

React applications are built using components, which are reusable blocks of code. Components help organize the user interface and improve code reusability.

Props (Properties) are used to pass data from a parent component to a child component. In this experiment, product details such as name, price, and category are passed from the App component to the ProductCard component using props.

The map() function is used to display multiple products dynamically from an array. When the Buy Now button is clicked, an alert message is displayed.

Algorithm

1. Create a React application.
2. Create a ProductCard component.
3. Store product details in an array.
4. Pass product data as props to the ProductCard component.
5. Use the map() function to display all products.
6. Add a Buy Now button.
7. Display an alert message when the button is clicked.

Folder Structure

product-list

```
|
├── src
|   ├── App.js
|   ├── ProductCard.js
|   ├── index.js
|   └── App.css
|
├── public
├── package.json
└── node_modules
```

Program

ProductCard.js

```
import React from "react";
function ProductCard({name, price, category }) {
  return (
    <div style={{border: "1px solid black", margin: "10px", pac
      <h3>{name}</h3>
      <p>Price: ₹{price}</p>
      <p>Category: {category}</p>
      <button onClick={() => alert(`${name} Purchased`)}>
        Buy Now
      </button>
    </div>
  );
}

export default ProductCard;
```

App.js

```
function App() {
 const products = [
  { id: 1, name: "Laptop", price: 50000, category: "Electroni
  { id: 2, name: "Mobile", price: 20000, category: "Electronic
  { id: 3, name: "Shoes", price: 1500, category: "Fashion" }
 ];

 return (
  <div>
   <h1>Product Listing</h1>

   {products.map((product) => (
    <ProductCard
     key={product.id}
     name={product.name}
     price={product.price}
     category={product.category}
    />
   ))}
```

```
    />
   ))}
  </div>
 );
}
export default App;
```

Sample Input

Product 1

Name : Laptop

Price : ₹50000

Category : Electronics

Product 2

Name : Mobile

Price : ₹20000

Category : Electronics

Sample Output

Product Listing

Laptop

Price : ₹50000

Category : Electronics

[Buy Now]

Mobile

Price : ₹20000

Category : Electronics

[Buy Now]

When Buy Now is clicked:

Laptop Purchased

(or the corresponding product name)

Result

Thus, the Product Listing Page using React Components and Props was developed and executed successfully.

Product Listing

Laptop

Price: ₹50000

Category: Electronics

Buy Now

Mobile

Price: ₹20000

Category: Electronics

Buy Now

3.Inventory Management Application Using React Hooks

Installation Procedure:

The setup process is identical to previous React experiments. A React project is created using Vite, dependencies are installed using npm, and the application is executed using the Vite development server. React Hooks(useState, useEffect, etc.) are included in React by default and require no additional installation.

Aim

To create a product listing page using multiple stateless React components and demonstrate the use of props for passing data and handling user interactions.

Software Requirements

- Node.js
- ReactJS
- VS Code
- Web Browser

Theory

React Hooks enable functional components to manage state and lifecycle features without using classes.

- useState manages component state.
- useEffect handles side effects such as Dom updates.
- useMemo optimizes performance by memorizing computed values.
- useRef provides direct access to DOM elements.
- useContext allows data sharing across components without prop drilling.

Procedure

1. Create a React application.
2. Create Header and ProductCard components.
3. Pass product details through props.
4. Display product name, price, and category.
5. Add a "Buy Now" button.
6. Run the application and verify the output.

Program

```
import React, { useState, useMemo } from "react";
export default function App() {
  const [products] = useState([
    { name: "Pen", qty: 10, price: 20 },
    { name: "Book", qty: 5, price: 100 }
  ]);

  const [search, setSearch] = useState("");
  const [dark, setDark] = useState(false);

  const totalValue = useMemo(() => {
    return products.reduce(
      (sum, item) => sum + item.qty * item.price,
      0
    );
  }, [products]);

  const filtered = products.filter((p) =>
    p.name.toLowerCase().includes(search.toLowerCase())
```

```
);

return (
  <div
    style={{
      backgroundColor: dark ? "#222" : "#fff",
      color: dark ? "#fff" : "#000",
      minHeight: "100vh",
      padding: "20px"
    }}
  >
    <h2>Inventory Management</h2>
    <button onClick={() => setDark(!dark)}>
      Toggle Theme
    </button>

    <br /><br />
```

```
    <input
      type="text"
      placeholder="Search Product"
      value={search}
      onChange={(e) => setSearch(e.target.value)}
    />

    <ul>
      {filtered.map((p, index) => (
        <li key={index}>
          {p.name} - Qty: {p.qty} - ₹{p.price}
        </li>
      ))}
    </ul>

    <h3>Total Value: ₹{totalValue}</h3>
  </div>
 );
}
```

Sample Input:

Enter product name and price,

click **Add Product**, and toggle theme.

Sample Output:

Displays product list,

total inventory value, and switches between light/dark themes.

Result:

Thus, a React application demonstrating React Hooks (useState, useEffect, useMemo, useRef, and useContext) was successfully developed and executed.

Inventory Management

Toggle Theme

Search Product

- Pen - Qty: 10 - ₹20
- Book - Qty: 5 - ₹100

Total Value: ₹700

4. Simple E-Commerce Application Using React Router

Installation

Node.js (Latest LTS version)

npm (Node Package Manager)

Visual Studio Code

ReactJS

Installation Steps

Step 1: Create a React Application

npx create-react-app ecommerce-app

Step 2: Navigate to the Project Folder

cd ecommerce-app

Step 3: Install React Router

npm install react-router-dom

Step 4: Start the React Application

npm start

The application opens in the browser at:

http://localhost:3000

Aim

To understand the concept of routing in React applications using React Router.

To implement navigation between different pages without reloading the browser.

To create multiple pages such as Home, Products, Cart, Contact, and Login.

To implement protected routes and dynamic routes in a React application.

Theory

React Router is a standard library used for routing in React applications. It enables navigation between different components without refreshing the entire web page, making the application faster and providing a better user experience.

Components Used

Browser Router: Wraps the entire application and enables routing.

Routes: Contains all route definitions.

Route: Maps a URL path to a React component.

Link: Creates navigation links without reloading the page.

useParams(): Reads dynamic values from the URL.

Protected Route: Restricts access to specific pages unless the user is authenticated.

Procedure

Create a React application using Create React App.

Install the react-router-dom package.

Create components:

Home

Products

Cart

Contact

Login

Import BrowserRouter, Routes, Route, and Link.

Configure navigation links.

Define routes for all pages.

Implement a protected route for authenticated users.

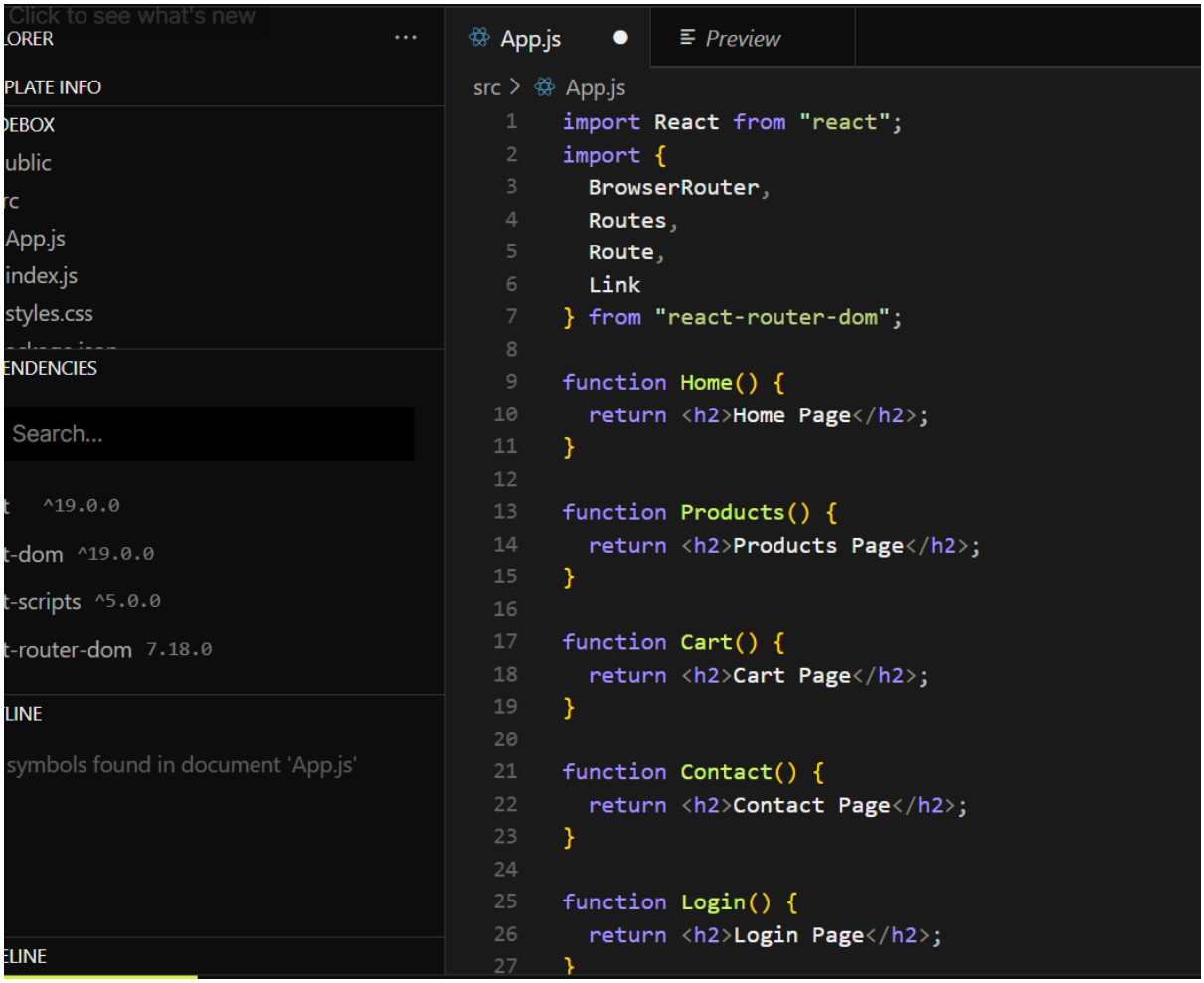
Implement a dynamic route using URL parameters.

Run the application using:

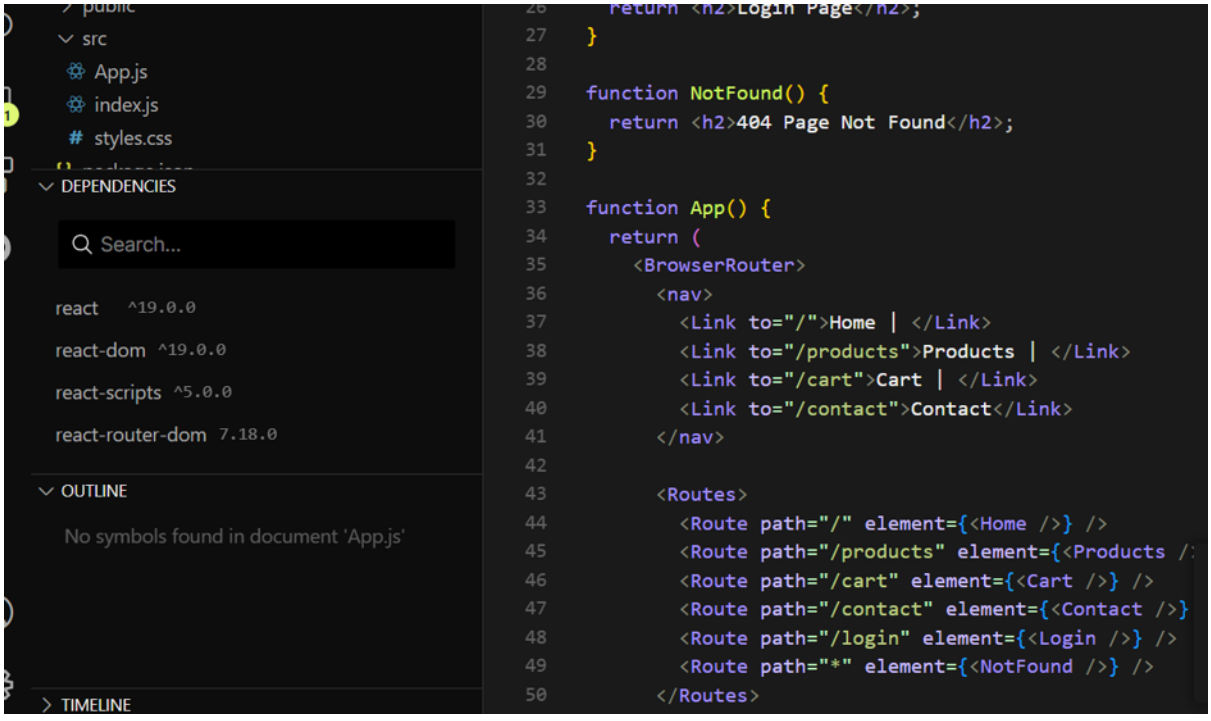
npm start

Verify navigation between pages.

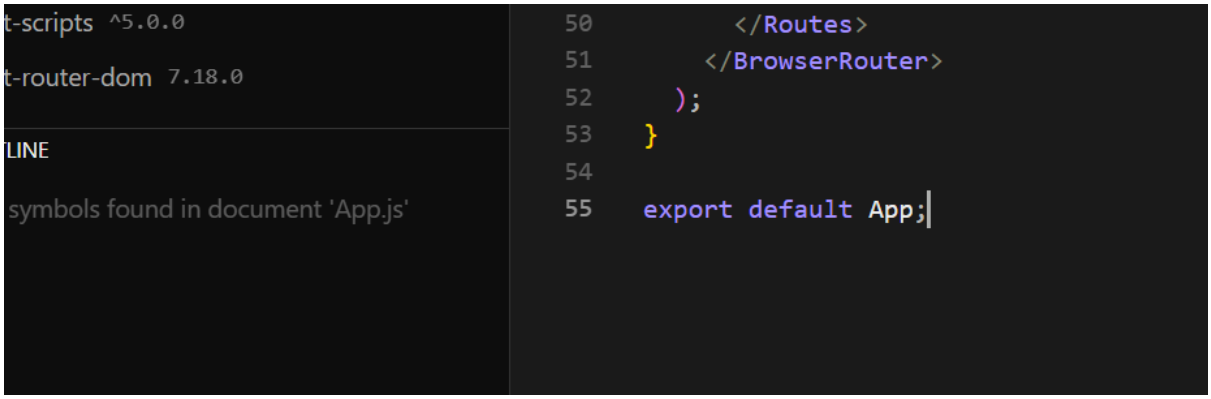
Program



```
import React from "react";
import {
  BrowserRouter,
  Routes,
  Route,
  Link
} from "react-router-dom";

function Home() {
  return <h2>Home Page</h2>;
}

function Products() {
  return <h2>Products Page</h2>;
}

function Cart() {
  return <h2>Cart Page</h2>;
}

function Contact() {
  return <h2>Contact Page</h2>;
}

function Login() {
  return <h2>Login Page</h2>;
}
```



```
function NotFound() {
  return <h2>404 Page Not Found</h2>;
}

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home | </Link>
        <Link to="/products">Products | </Link>
        <Link to="/cart">Cart | </Link>
        <Link to="/contact">Contact</Link>
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/products" element={<Products />
        <Route path="/cart" element={<Cart />} />
        <Route path="/contact" element={<Contact />}
        <Route path="/login" element={<Login />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
```



Sample Input

User Actions

Click Home

Click Products

Click Cart

Click Contact

Click Login

Open URL:

/products/101

Sample Output

Home Page

Home Page

Products Page

Products Page

Cart Page

Cart Page

Contact Page

Contact Page

Login Page

Login Page

Dynamic Route

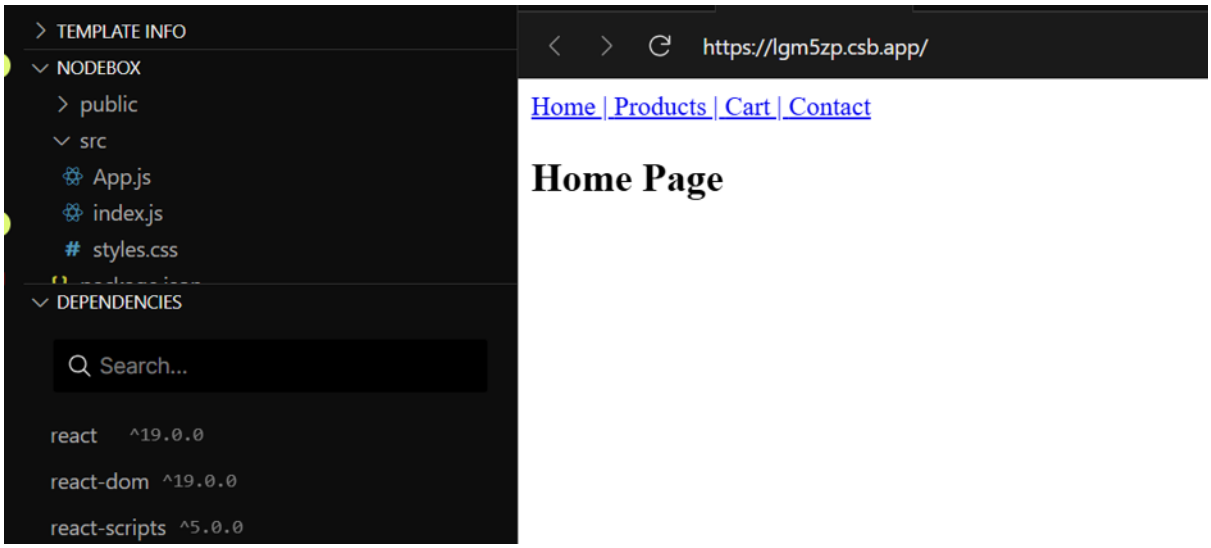
Product ID : 101

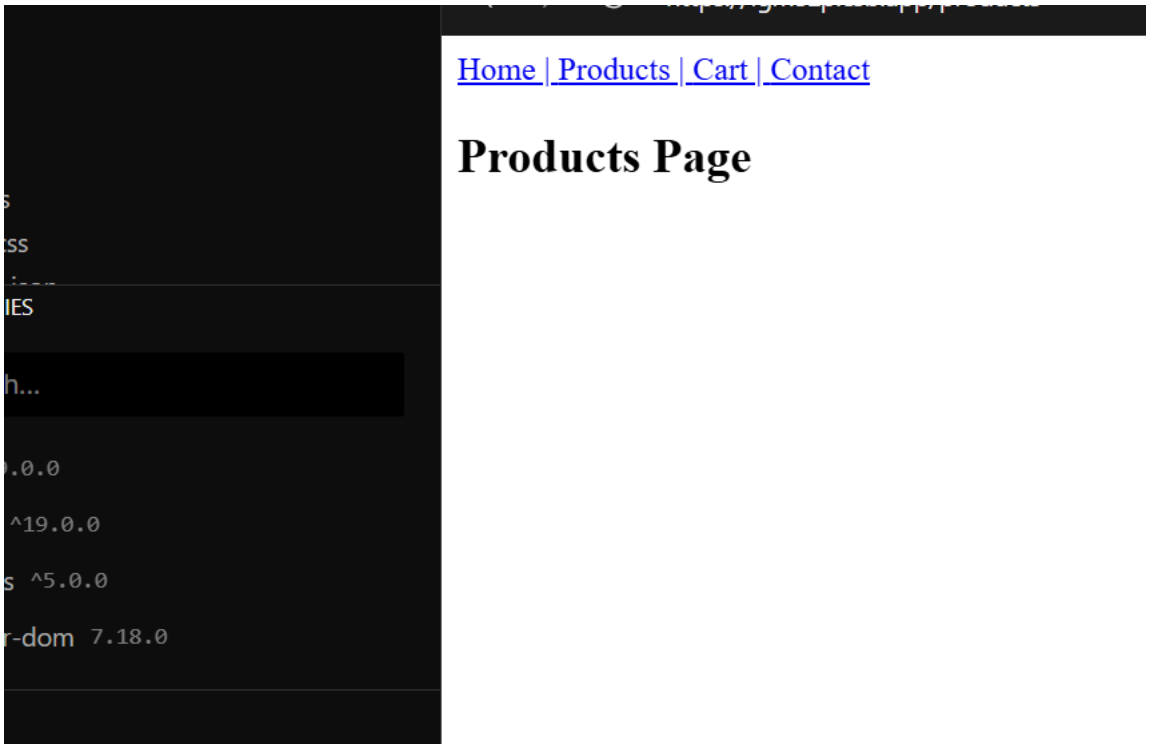
Invalid URL

404 Page Not Found

Result

Thus, a simple E-Commerce application was successfully developed using React Router. The application demonstrated client-side routing, navigation between multiple pages without page refresh, implementation of protected routes, dynamic routing using URL parameters, and handling of invalid URLs through a 404 page**.**





5. Dynamic React Form Using Configuration Object

Installation

Prerequisites

- Node.js (Latest LTS Version)
- npm (Node Package Manager)
- Visual Studio Code
- ReactJS

Installation Steps

Step 1: Create a React Application

npx create-react-app dynamic-form-app

Step 2: Navigate to the Project Folder

cd dynamic-form-app

Step 3: Start the React Application

npm start

The application opens in the browser at:

http://localhost:3000

Aim

1. To generate form fields dynamically using a JavaScript configuration object.
2. To manage form data efficiently using React state.
3. To display user-entered data dynamically after form submission.
4. To understand dynamic form rendering and event handling in React.

Theory

A Dynamic React Form is a form whose input fields are generated automatically from a JavaScript configuration object instead of being written manually. This approach makes forms reusable, scalable, and easier to maintain.

A configuration object is usually an array of objects where each object represents one form field and contains properties such as field name, label, input type, and placeholder.

React uses the map() function to iterate through the configuration object and generate the required input elements dynamically.

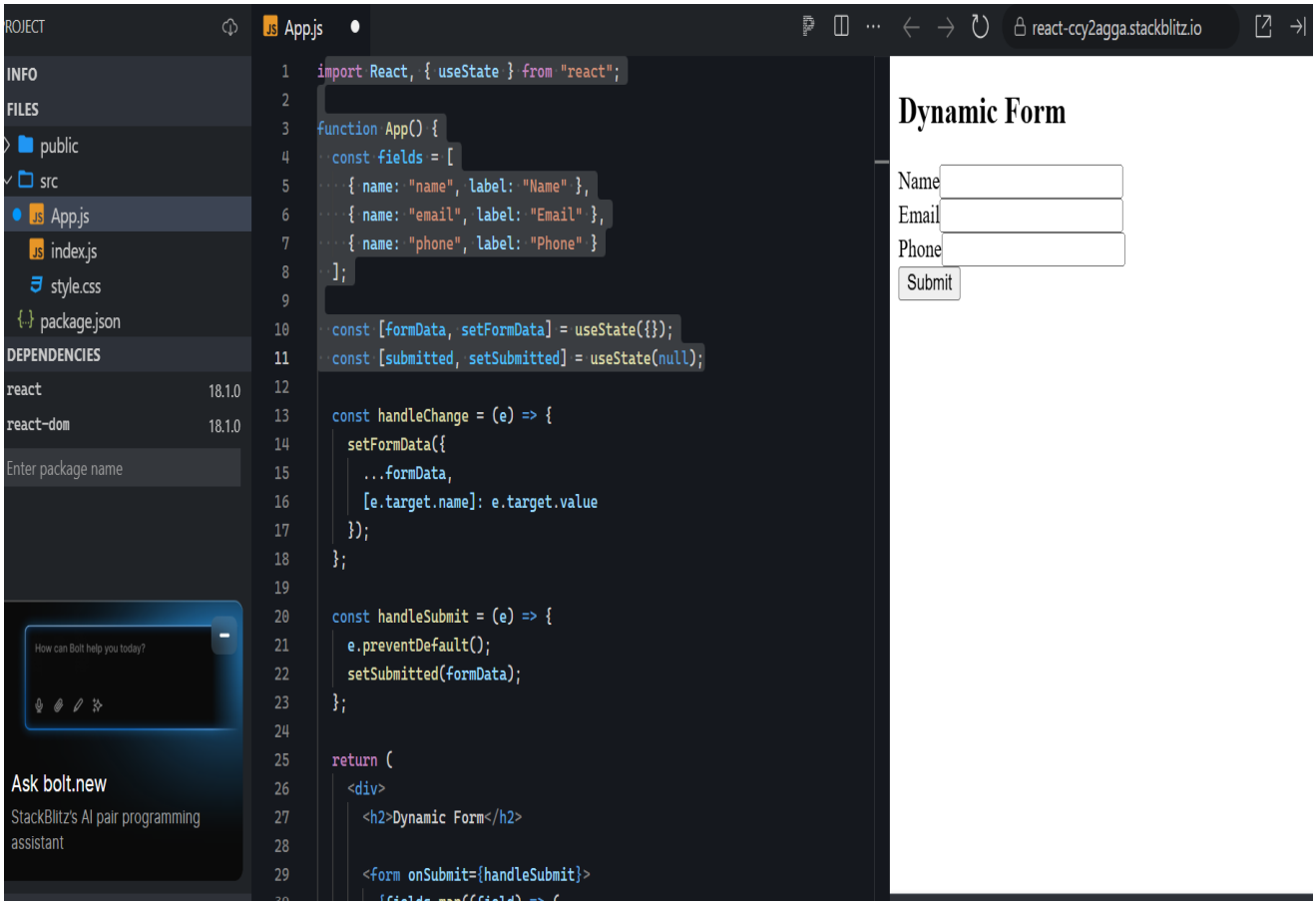
The useState() hook is used to store and update the values entered by the user. When the form is submitted, the entered data is displayed below the form without refreshing the page.

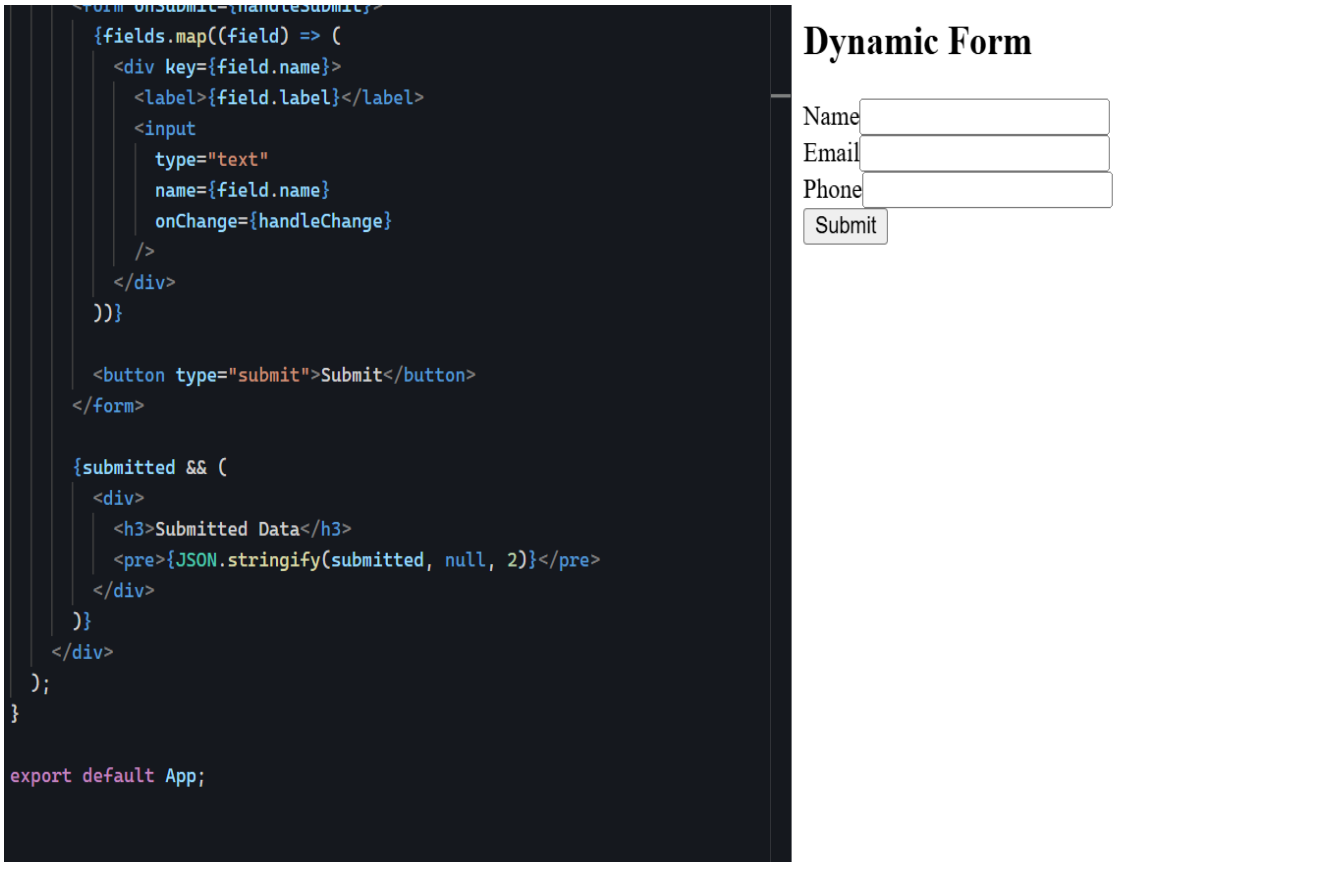
Procedure

1. Create a React application using Create React App.
2. Create a JavaScript configuration object containing the form field details.
3. Use the map() method to dynamically generate form fields.
4. Initialize state variables using the useState() hook.
5. Capture user input using the onChange event.
6. Store the entered values in the state object.
7. Add a Submit button to the form.
8. Display the submitted form data below the form.
9. Run the application using:

   npm start
10. Verify that all fields are generated dynamically and the submitted data is displayed correctly.

Program


PROJECT
INFO
FILES
public
src
App.js
index.js
style.css
package.json
DEPENDENCIES
react 18.1.0
react-dom 18.1.0
Enter package name
How can Bolt help you today?
Ask bolt.new
StackBlitz's AI pair programming assistant
App.js
react-ccy2agga.stackblitz.io
1 import React, { useState } from "react";
2
3 function App() {
4 const fields = [
5 { name: "name", label: "Name" },
6 { name: "email", label: "Email" },
7 { name: "phone", label: "Phone" }
8];
9
10 const [formData, setFormData] = useState({});
11 const [submitted, setSubmitted] = useState(null);
12
13 const handleChange = (e) => {
14 setFormData({
15 ...formData,
16 [e.target.name]: e.target.value
17 });
18 };
19
20 const handleSubmit = (e) => {
21 e.preventDefault();
22 setSubmitted(formData);
23 };
24
25 return (
26
27 <h2>Dynamic Form</h2>
28
29 <form onSubmit={handleSubmit}>
Dynamic Form
Name
Email
Phone
Submit


{fields.map((field) => (

<label>{field.label}</label>
<input
type="text"
name={field.name}
onChange={handleChange}
/>

))}
<button type="submit">Submit</button>
</form>
{submitted && (

<h3>Submitted Data</h3>
<pre>{JSON.stringify(submitted, null, 2)}</pre>

)}

);
}
export default App;
Dynamic Form
Name
Email
Phone
Submit

Sample Input

User enters the following details:

Field	Value
Name	John Smith
Email	john@example.com
Mobile Number	9876543210
Age	22
City	Hyderabad

Then clicks the Submit button.

Sample Output

Dynamic Form

Registration Form

Name : _____

Email : _____

Mobile Number : _____

Age : _____

City : _____

[Submit]

After Clicking Submit

Submitted Details

Name : John Smith

Email : john@example.com

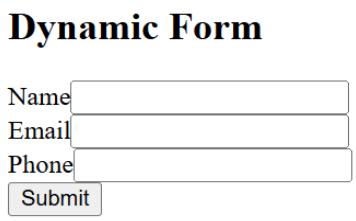
Mobile Number : 9876543210

Age : 22

City : Hyderabad

Result

Thus, a Dynamic React Form was successfully developed using a JavaScript configuration object. The form fields were generated dynamically, user inputs were managed using React state, and the submitted data was displayed below the form without reloading the page.



6. React SPA with Routing and API Fetching

Installation

Step 1: Create a React Application

npx create-react-app react-spa-api

Step 2: Navigate to the Project Folder

cd react-spa-api

Step 3: Install React Router

npm install react-router-dom

Step 4: Start the React Application

npm start

The application opens in the browser:

http://localhost:3000

Aim

To develop a React Single Page Application (SPA) using React Router and integrate it with a public API to fetch and display user data dynamically.

Software Requirements

- Operating System: Windows 10/11 or Linux
- Node.js (Latest LTS Version)
- npm (Node Package Manager)
- Visual Studio Code
- React.js
- React Router DOM
- Internet Connection (to access the public API)

Theory

A Single Page Application (SPA) is a web application that loads a single HTML page and updates the content dynamically without refreshing the browser. This approach provides faster navigation and a better user experience.

React Router enables client-side routing, allowing users to navigate between pages such as Home and Users without reloading the application.

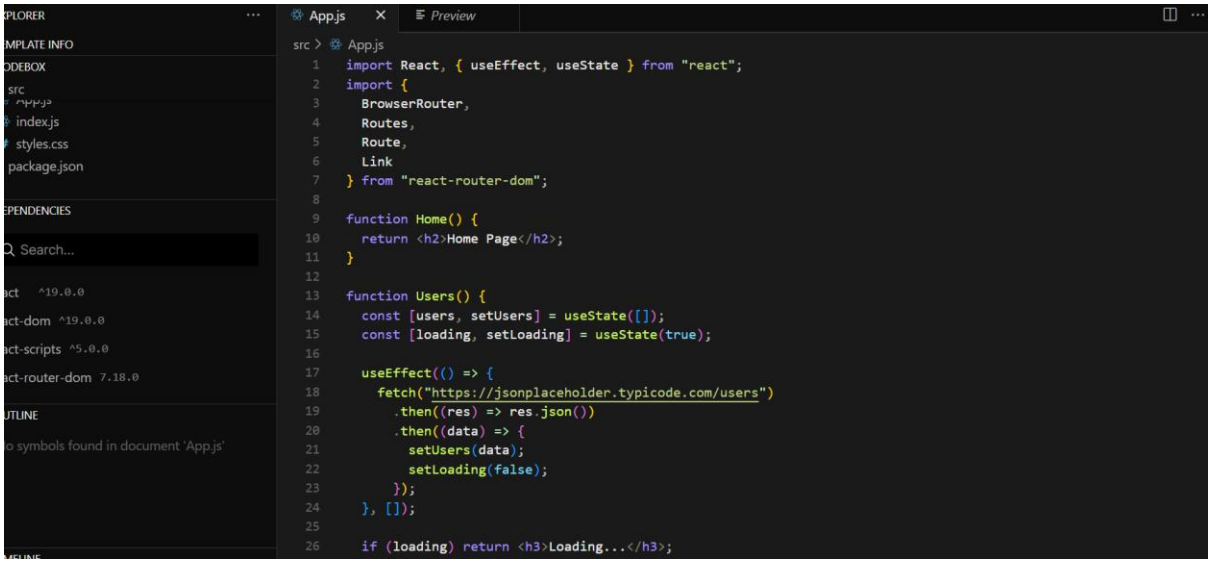
React provides the fetch() API to retrieve data from web services. The useEffect() hook is used to perform API calls after the component is rendered, while useState() stores the fetched data and updates the user interface automatically.

During the API request, a loading message is displayed to inform users that data is being retrieved. Once the response is received, the user details are displayed on the screen.

Algorithm / Procedure

1. Create a React application using Create React App.
2. Install the React Router DOM package.
3. Create Home and Users components.
4. Configure routing using BrowserRouter, Routes, and Route.
5. Use fetch() to retrieve user data from a public API.
6. Store the fetched data using the useState() hook.
7. Use the useEffect() hook to call the API after the component loads.
8. Display a loading message while the data is being fetched.
9. Display the list of users after successful retrieval.
10. Run the application and verify navigation and API integration.

Program




```
  return (
    <div>
      <h2>Users List</h2>

      <ul>
        {users.map((user) => (
          <li key={user.id}>
            {user.name}
          </li>
        ))}
      </ul>
    </div>
  );
}

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home | </Link>
        <Link to="/users">Users</Link>
      </nav>
```

```
50
51          <Routes>
52            <Route path="/" element={<Home />} />
53            <Route path="/users" element={<Users />} />
54          </Routes>
55        </BrowserRouter>
56      );
57    }
58
59    export default App;
```

Sample Input

Navigate to:

http://localhost:3000/users

The application fetches data from:

https://jsonplaceholder.typicode.com/users

Sample Output

Users List

Leanne Graham

Ervin Howell

Clementine Bauch

Patricia Lebsack

Chelsey Dietrich

Mrs. Dennis Schulist

Kurtis Weissnat

Nicholas Runolfsdottir V

Glenna Reichert

Clementina DuBuque

While data is loading:

Loading...

Result

Thus, a React Single Page Application (SPA) with React Router and API fetching was successfully developed. The application enabled seamless navigation between pages, retrieved user data from a public API using the fetch() method, and displayed the data dynamically without refreshing the page.

Home | Users

Home Page

Home | Users

Users List

- Leanne Graham
- Ervin Howell
- Clementine Bauch
- Patricia Lebsack
- Chelsey Dietrich
- Mrs. Dennis Schulist
- Kurtis Weissnat
- Nicholas Runolfsdottir V
- Glenna Reichert
- Clementina DuBuque